

Software Engineering



Dr. Fatma Eskander
Math.department, Faculty of Science, Mansoura
University



A brief Introduction to Testing

22/11/2025

Introduction

What is Software Testing?

- ❑ The process of checking if your software works as expected and is free of bugs.

Why Do We Test?

- ❑ **Find Bugs Early:** Cheaper to fix now than later.
- ❑ **Ensure Quality:** Deliver a reliable and usable product.
- ❑ **Build Confidence:** Trust that your software works correctly.

Testing in Agile vs. Plan-Driven Development

Plan-Driven (Waterfall):

Testing is a separate phase at the **end**.

Risk: Finding critical bugs late is very expensive.

Agile:

□ Testing is **continuous**, integrated throughout development.

Benefit: Catches issues early and reduces cost of change.

Agile

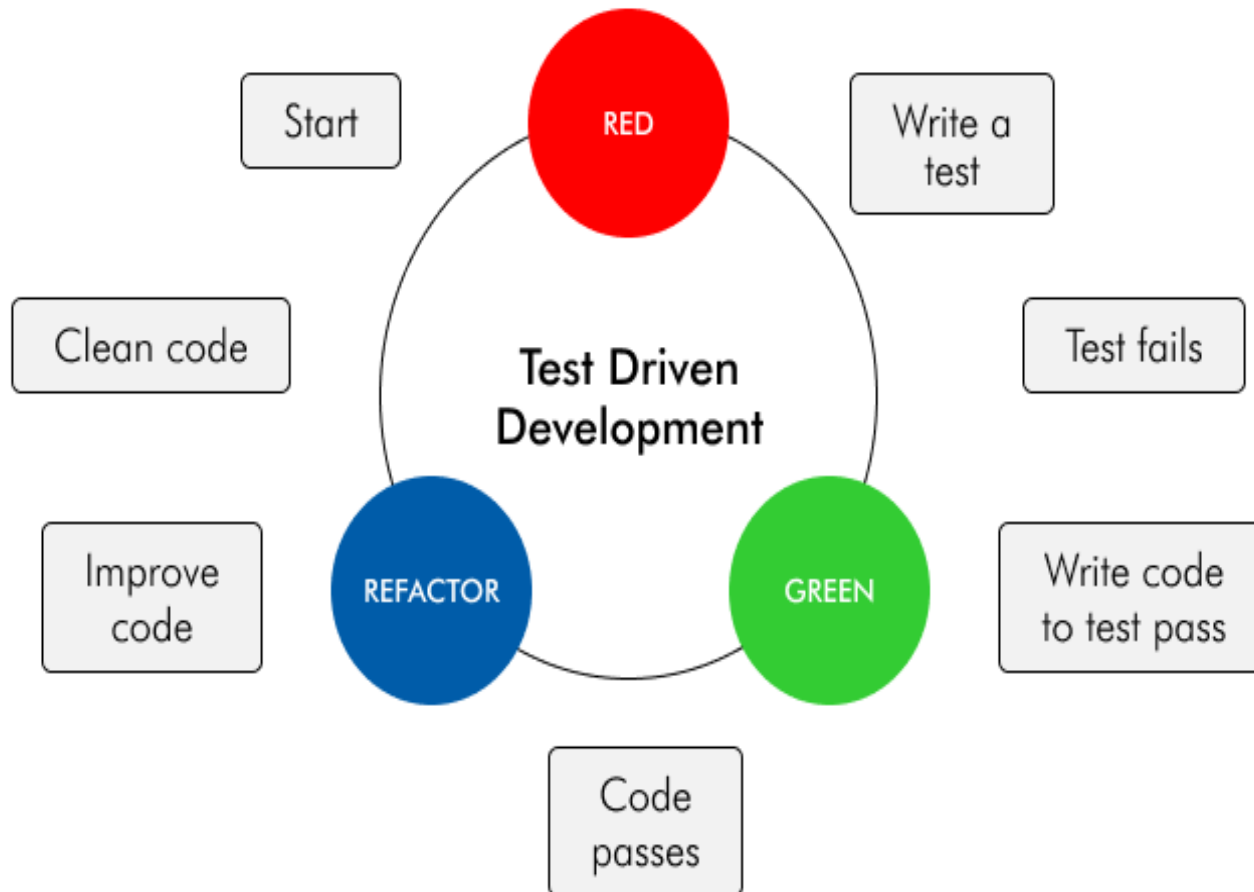
How Agile Promotes Continuous Testing:

- ❑ **Iterative Development:** Build and test in small, repeated cycles.
- ❑ **Test-Driven Development (TDD):** Write tests before writing the code.

Test-Driven Development (TDD)

❑ The "Red-Green-Refactor" Cycle

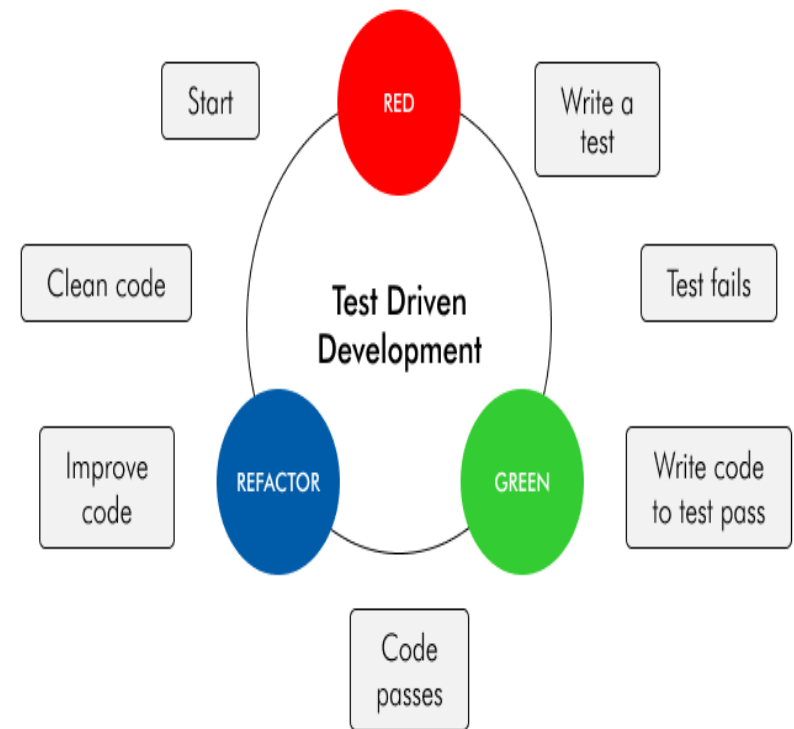
- ❑ Like writing a to-do list before you start a task.



Test-Driven Development (TDD)

□ The "Red-Green-Refactor" Cycle

1. **Write a Test (Red):** Write a small test for a new function. It should **FAIL** initially (**Red**).
2. **Write Minimal Code (Green):** Write **just enough** code to make the test pass (**Green**).

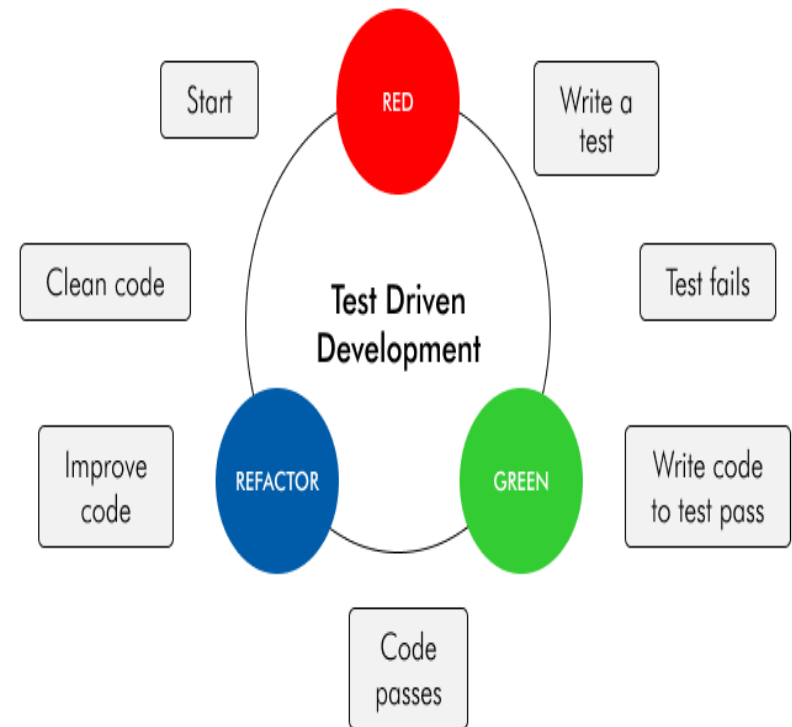


Test-Driven Development (TDD)

□ The "Red-Green-Refactor" Cycle

3. **Refactor:** Clean up the code while keeping the test green. Improve its structure and readability.

4. **Repeat:** Start the cycle over for the next piece of functionality.

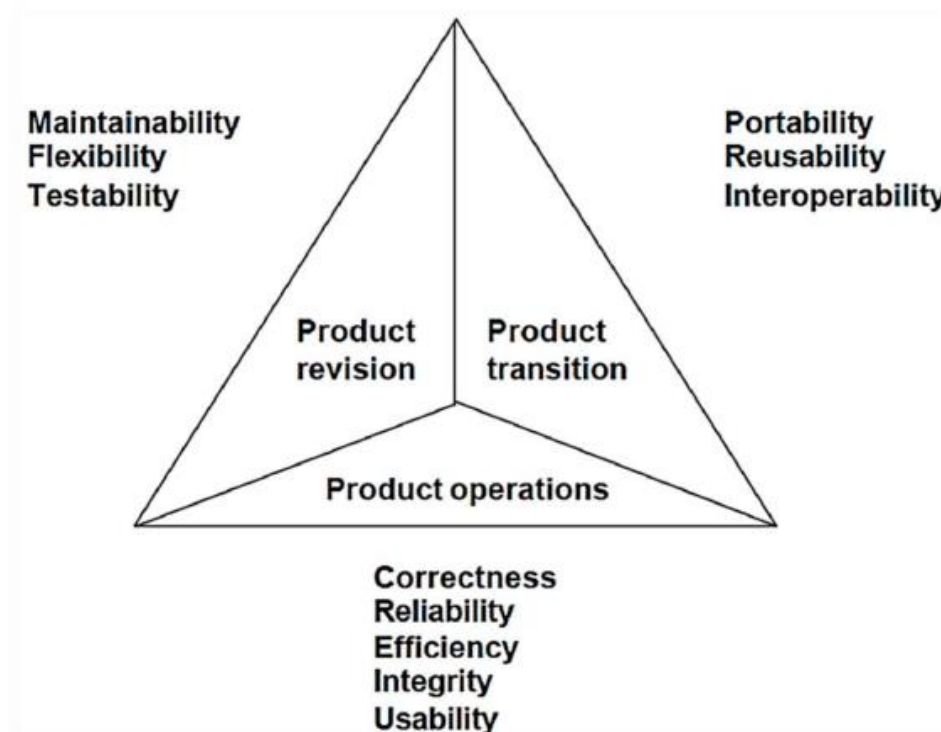


What Makes Software "Good"?

❑ Software Quality Factors

Quality is more than "it works."

Three Main Categories:



What Makes Software "Good"?

❑ Software Quality Factors

A- Product Operation (How it runs for the end-user):

- ❑ **Correctness:** Does it perform the exact tasks defined in the requirements?
- ❑ **Reliability:** Does it work consistently without unexpected failures?
- ❑ **Efficiency:** How well does it use computing resources (CPU, memory, time)?

What Makes Software "Good"?

❑ Software Quality Factors

A- Product Operation (How it runs for the end-user):

- ❑ **Integrity:** Is it secure? Can it control access to data and prevent unauthorized use?
- ❑ **Usability:** Is it easy to learn, operate, and understand?

What Makes Software "Good"?

❑ Software Quality Factors

B- Product Revision (How easy it is to maintain and improve):

- ❑ **Maintainability:** How easily can we find and fix a bug?
- ❑ **Testability:** How easy is it to create tests for the software?
- ❑ **Flexibility:** How easy is it to modify the code to add new features or change existing ones?

What Makes Software "Good"?

❑ Software Quality Factors

C- Product Transition (How easy it is to move to new environments):

- ❑ **Portability:** Can we easily make it run on a different operating system or platform?
- ❑ **Reusability:** Can we use parts of this code in other projects?
- ❑ **Interoperability:** How easily can this system exchange data and work with other systems?

Trade-offs Between Quality Factors

❑ Some pairs are not independent

Improving one quality factor can often negatively impact another. Design is about making intelligent trade-offs.

❑ Common Trade-offs:

1- Security vs. Performance (Integrity & Efficiency)

Adding strong password encryption (**Integrity**) can slow down login times (**Efficiency**).

Trade-offs Between Quality Factors

❑ Common Trade-offs:

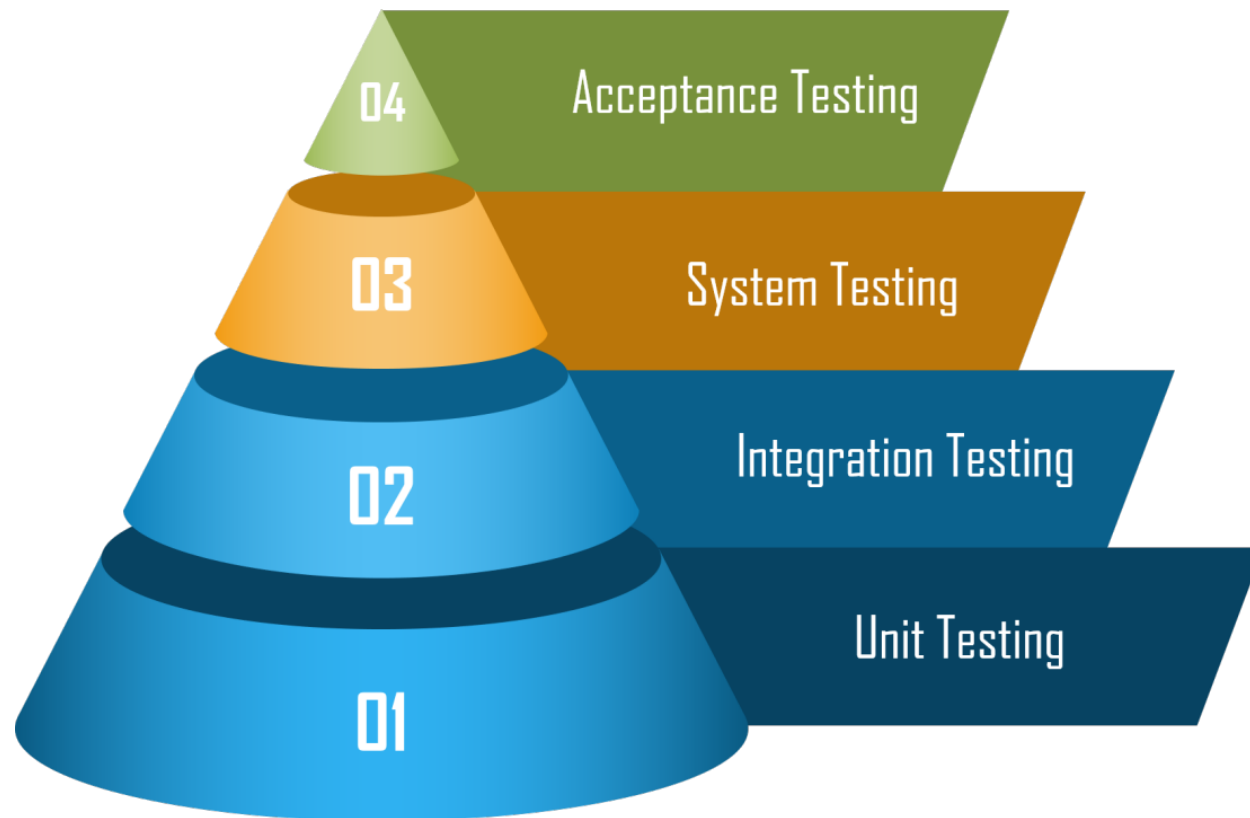
2- Usability vs. Portability

An app that uses unique MacOS features for a great experience (**Usability**) is hard to port to Windows (**Portability**).

❑ **Key Message:** You often have to balance competing quality factors.

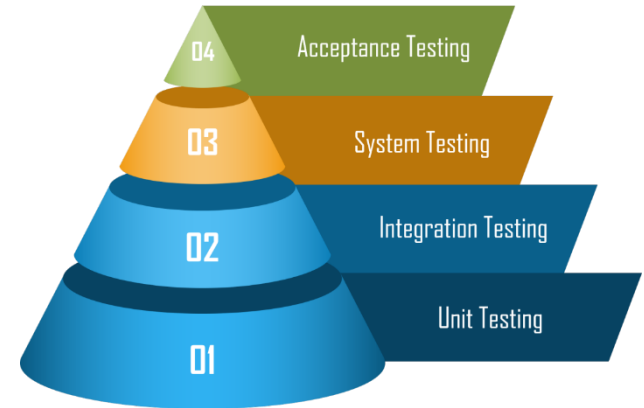
The Four Levels of Testing (The Testing Pyramid)

- ❑ Use a pyramid diagram.



The Four Levels of Testing (The Testing Pyramid)

1- Unit Testing: Testing individual functions or classes in isolation.



2- Integration Testing : Testing how multiple units work together.

3- System Testing: Testing the complete, integrated system against requirements.

4- Acceptance Testing: Final testing to ensure it meets the user's needs.

How to Create Test Cases - Black-Box vs. White-Box



Black-Box Testing (The User's View):

I don't see the code. I test the functionality based on requirements.

How to Create Test Cases - Black-Box vs. White-Box



ZERO KNOWLEDGE
Testing as a User



SOME KNOWLEDGE



FULL KNOWLEDGE
Testing as a Developer

White-Box Testing (The Developer's View):

I see the code. I test the internal structures and logic.

How to Create Test Cases - Black-Box vs. White-Box

❑ **Black-Box Technique: Equivalence Partitioning & Boundary Testing**

❑ **The Problem:** It's impossible to test every single input.

❑ **The Solution:**

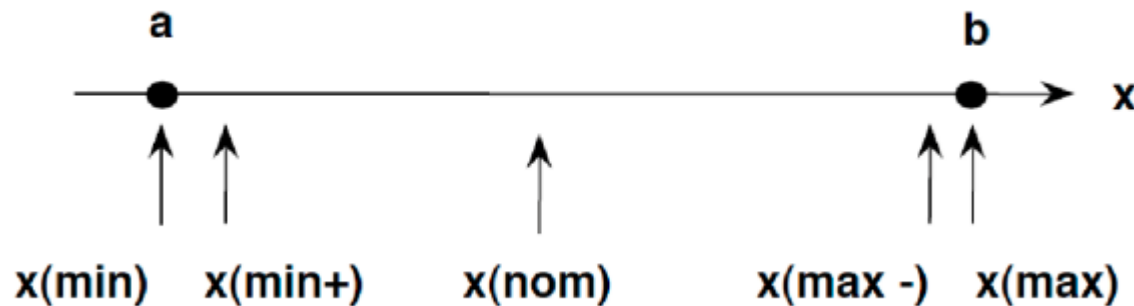
1. Equivalence Partitioning: Group inputs that should be treated the same.

Example: For an "Age" field (13-19 for a teen discount), valid groups are: `<<13`, `13-19`, `<>19`.

How to Create Test Cases - Black-Box vs. White-Box

2. Boundary Value Analysis: Test at the edges of these groups.

Example: Test ages 12, 13, 19, and 20. Bugs love to hide at boundaries!



How to Create Test Cases - Black-Box vs. White-Box

- ❑ **White-Box Technique: Path Testing**
- ❑ **Goal:** Ensure every possible path through the code is tested at least once.
- ❑ **How?** Create a **Control Flow Graph** (showing decisions and paths).
- ❑ **Challenge:** Complex code with many `if` statements and loops has **many** paths.
- ❑ **Solution:** Use **Cyclomatic Complexity** to count the number of independent paths we need to test.

Measuring Complexity: Cyclomatic Complexity (CCM)

- ❑ **What it is:** A metric that measures how complex a piece of code is.
- ❑ **How it's calculated:** Count the decisions ('if', 'while', 'for', '&&', '||') in the code and add 1.
- ❑ **Why it matters:**
High CCM => More complex code => Harder to test and maintain.
Low CCM => Simpler, more linear code => Easier to test.

Measuring Complexity: Cyclomatic Complexity (CCM)

❑ Example:

Line number	Code
1	public static boolean equals (long[] a, long[] a2) {
2	if (a==a2)
3	return true;
4	if (a==null a2==null)
5	return false;
6	int length = a.length;
7	if (a2.length != length)
8	return false;
9	for (int i=0; i<length; i++)
10	if (a[i] != a2[i])
11	return false;
12	return true;
13	}

Cyclomatic Complexity metric (CCM): given by counting the number of **independent paths** through a method body.

CCM = 7